

Documentation Technique — Système IAM / PAM

Projet de Fin d'Études — ESPRIT

Auteur : Omar Ben Tahar

Date : Mai 2026

Table des matières

1. Vue d'ensemble du projet
 2. Architecture globale
 3. Glossaire technique
 4. Infrastructure Docker
 5. Backend Spring Boot
 6. Frontend Angular
 7. Flux de données par fonctionnalité
 8. Stockage des données
-

1. Vue d'ensemble du projet

1.1 Problématique

Les entreprises disposant de systèmes d'information critiques (banques, fintechs, opérateurs) font face à deux risques majeurs :

- **Accès non contrôlé** : n'importe quel administrateur peut se connecter à un serveur de production sans traçabilité.
- **Prolifération des identités** : plusieurs annuaires (Active Directory, LDAP), plusieurs applications, des mots de passe différents partout.

1.2 Ce que fait le système

Le projet implémente une plateforme **IAM/PAM multi-tenant** combinant :

Capacité	Description
IAM (Identity & Access Management)	Gestion centralisée des identités via Keycloak + LDAP
PAM (Privileged Access Management)	Contrôle d'accès aux ressources critiques (SSH, RDP, BDD, Web, API)
MFA (Multi-Factor Authentication)	TOTP, e-mail OTP, SMS, WhatsApp
Face ID	Authentification biométrique via descripteur facial
Audit & Conformité	Enregistrement de chaque action, export CSV
Multi-tenant	Plusieurs organisations isolées sur une même instance

1.3 Acteurs du système

Rôle	Description
<code>admin</code>	Super-administrateur de la plateforme. Crée et gère les tenants.
<code>tenant-admin</code>	Administrateur d'une organisation. Gère ses utilisateurs, ressources et demandes d'accès.
<code>user</code>	Utilisateur final. Soumet des demandes d'accès, ouvre des sessions.
<code>pam-access</code>	Variante de <code>user</code> avec accès PAM direct.
<code>auditor</code>	Auditeur. Consulte les journaux en lecture seule.

1.4 Flux métier principal

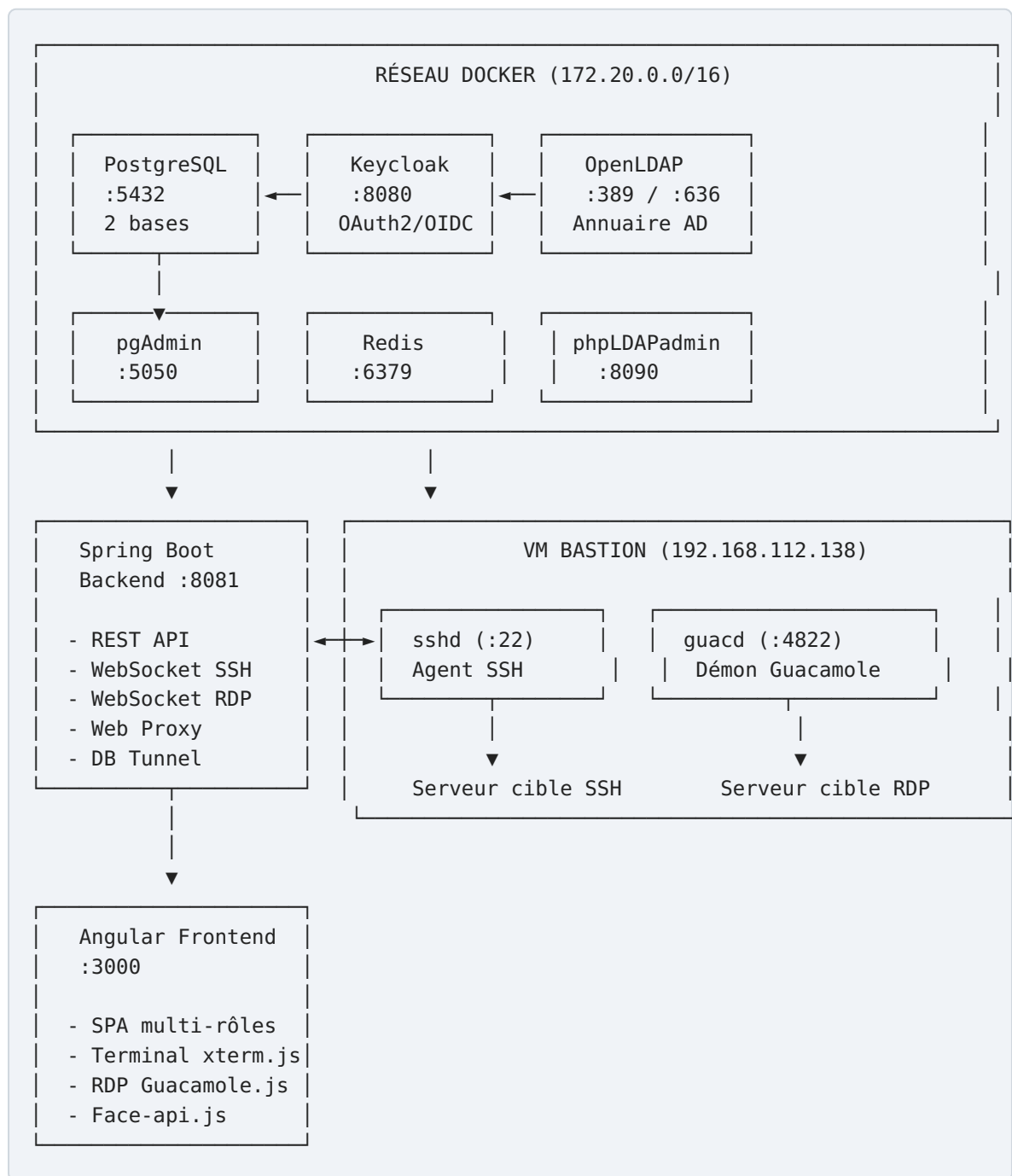
```

Utilisateur → demande d'accès → tenant-admin approuve → utilisateur ouvre la session
                                                    ↓
SSH → bastion VM → cible
RDP → guacd → cible
Web → proxy HTTP → cible
BDD → tunnel TCP → cible

```

2. Architecture globale

2.1 Vue de haut niveau



2.2 Couches applicatives

Couche	Technologie	Responsabilité
Présentation	Angular 15 + Angular Material	UI multi-rôles, SPA
API Gateway	Spring Boot 3 (Tomcat intégré)	REST + WebSocket

Couche	Technologie	Responsabilité
Sécurité	Spring Security + Keycloak	OAuth2 Resource Server, JWT
Domaine	Services Spring (@Service)	Logique métier
Persistance	Spring Data JPA + PostgreSQL	ORM, requêtes
Accès distant	Apache SSHD + Guacamole Client	Tunneling SSH/RDP
Identité	Keycloak 23	SSO, fédération LDAP
Notification	JavaMail + Twilio	E-mail/SMS/WhatsApp OTP

2.3 Multi-tenancy

Chaque organisation (tenant) est isolée par :

1. **Schéma PostgreSQL dédié** : `tenant_bank_a` , `tenant_bank_b` , `tenant_fintech_c`
2. **Groupe Keycloak** : chaque utilisateur appartient au groupe `/tenantId`
3. **Claim JWT groups** : contient l'identifiant du tenant
4. **TenantContext (ThreadLocal)** : propagé à tous les services pour chaque requête

3. Glossaire technique

3.1 JWT (JSON Web Token)

Un JWT est un jeton signé en trois parties : `header.payload.signature` .

- **Header** : algorithme de signature (`RS256` pour Keycloak, `HS256` pour Face Auth)
- **Payload (claims)** : données utiles — `sub` (sujet), `preferred_username` , `realm_access.roles` , `groups` , `exp`
- **Signature** : garantit l'intégrité

Le backend valide le JWT à chaque requête via `JwtDecoder` . Il ne maintient **aucune session** (mode `STATELESS`).

Deux émetteurs coexistent :

- **Keycloak** : signé avec RSA, clé publique exposée sur `/.well-known/jwks.json`
- **Face Auth** : signé avec HMAC-256, clé partagée configurée dans `face.jwt.secret`

3.2 OAuth2 / OpenID Connect (OIDC)

OAuth2 est un protocole d'autorisation. **OIDC** ajoute une couche d'identité sur OAuth2.

Le flux utilisé est **Authorization Code Flow** :

1. L'Angular redirige l'utilisateur vers Keycloak
2. Keycloak authentifie (login/mot de passe ou LDAP)
3. Keycloak redirige avec un `code` d'autorisation
4. L'Angular échange le `code` contre un `access_token` (JWT) et `id_token`
5. L'Angular envoie l' `access_token` dans chaque requête HTTP (`Authorization: Bearer ...`)

3.3 Keycloak

Keycloak est un serveur IAM open-source qui gère :

- **Realm** : espace d'identité isolé (`iam-pam-realm`)
- **Clients** : applications enregistrées (`iam-pam-backend` , `iam-pam-frontend`)
- **Groupe**s : correspondent aux tenants de la plateforme
- **Rôles** : `admin` , `tenant-admin` , `user` , `pam-access` , `auditor`
- **Fédération d'identité** : synchronisation avec OpenLDAP (Active Directory)

3.4 LDAP / Active Directory

LDAP (Lightweight Directory Access Protocol) est un protocole d'accès à un annuaire d'utilisateurs.

L'OpenLDAP simulé représente un Active Directory d'entreprise :

- Domaine : `bank-a.local`
- Arborescence : `dc=bank-a,dc=local`
- Les utilisateurs LDAP sont synchronisés dans Keycloak via **User Federation**

Le backend peut aussi interroger l'AD directement via `LdapService` pour créer/importer des utilisateurs.

3.5 PAM (Privileged Access Management)

Le PAM contrôle l'accès aux ressources critiques selon le modèle **JIT (Just-In-Time)** :

- L'accès n'est pas permanent — il faut **demander** et attendre **approbation**
- L'accès a une **durée limitée** (ex : 4 heures)
- Tout est **tracé** dans les journaux d'audit

3.6 Bastion Host

Un bastion est un serveur intermédiaire qui **centralise** tous les accès SSH/RDP.

```
Utilisateur → bastion (:22) → serveur cible
```

Avantages :

- Seul le bastion est exposé sur le réseau
- Les serveurs cibles ne sont jamais directement accessibles depuis Internet
- Toutes les commandes passent par le bastion, donc traçables

Dans ce projet, le bastion tourne sur la VM 192.168.112.138 . Le backend s'y connecte en SSH avec une **clé privée** (bastion_key) puis lance une commande SSH vers la cible.

3.7 SSH Tunnel

SSH (Secure Shell) est un protocole de connexion chiffrée à distance.

Le flux du tunnel SSH de ce projet :

```
Browser (xterm.js) ↔ WebSocket ↔ Spring Boot ↔ Apache SSHD ↔ Bastion VM ↔ Cible SSH
```

- Apache SSHD ouvre une session SSH vers le bastion avec une **clé privée RSA**
- Le bastion exécute `ssh user@cible` ou `sshpass -p password ssh user@cible`
- Les frappes clavier de l'utilisateur sont transmises via WebSocket → STDIN du shell SSH
- La sortie SSH est relayée en sens inverse via STDOUT → WebSocket → xterm.js

3.8 WebSocket

HTTP est unidirectionnel (requête → réponse). WebSocket est un **canal bidirectionnel persistant** sur la même connexion TCP.

Utilisé ici pour :

- **Terminal SSH** : `ws://backend/ws/session/{requestId}?token=<jwt>`
- **Viewer RDP** : `ws://backend/ws/rdp/{requestId}?token=<jwt>`

Le JWT ne peut pas être envoyé dans un header HTTP lors de l'upgrade WebSocket (limitation du navigateur), donc il est passé en **query parameter**.

3.9 Guacamole / RDP

RDP (Remote Desktop Protocol) est le protocole de bureau à distance de Microsoft.

Apache Guacamole est une passerelle web pour RDP/VNC/SSH :

- Un démon **guacd** (C) parle RDP natif vers la cible
- Le frontend utilise **guacamole-common-js** pour afficher le bureau dans le navigateur
- Le backend relaie le protocole Guacamole (texte) entre le WebSocket et guacd via TCP :4822

Flux :

```
Browser (Guacamole.js) ↔ WebSocket ↔ Spring Boot ↔ TCP:4822 ↔ guacd ↔ RDP ↔ Cible Windows
```

3.10 MFA (Multi-Factor Authentication)

L'authentification multi-facteurs exige **deux preuves d'identité** :

1. Ce que je sais : mot de passe
2. Ce que je possède : téléphone (SMS/TOTP), e-mail

TOTP (Time-based OTP) : Algorithme RFC 6238. Un code à 6 chiffres calculé à partir d'un **secret partagé** et du **temps courant** (fenêtre de 30 secondes). Utilisé par Google Authenticator.

Implémentation dans MfaService :

```
code = HMAC-SHA1(secret, floor(epoch/30)) → troncature → 6 chiffres
```

OTP e-mail/SMS/WhatsApp : Code aléatoire à 6 chiffres envoyé par canal externe, valable 5 minutes.

3.11 Face Auth (Reconnaissance faciale)

Basé sur **face-api.js** (TensorFlow.js) :

- **Enrollment** : capture vidéo → détection du visage → calcul du descripteur (vecteur Float32 de 128 dimensions) → stocké en base
- **Vérification** : nouveau descripteur → distance euclidienne avec le descripteur stocké → `match` si distance < seuil (~0.5)

Le backend génère un JWT signé HS256 avec les claims `username`, `roles`, `tenantId` et `iss: "face-auth"`. Spring Security le valide comme n'importe quel autre JWT.

3.12 AES-256-GCM (Chiffrement des credentials)

Les mots de passe et clés privées des ressources PAM sont chiffrés **au repos** dans PostgreSQL via `AesEncryptionConverter` (`JPA @Converter`) :

- Algorithme : **AES-256-GCM** (authenticated encryption)
- Clé : 32 octets, configurée dans `resource.encryption.key`
- Le champ en base contient : `IV` (12 bytes) + `Ciphertext` + `Auth Tag` (16 bytes) , encodé Base64

4. Infrastructure Docker

4.1 Fichier `docker-compose.yml`

Réseau : `iam-pam-network` (bridge, subnet `172.20.0.0/16`)

Tous les services sont sur le même réseau interne. Chaque service est joignable par son **nom de service** DNS (`postgres` , `keycloak` , `openldap` ...).

4.2 Service `postgres`

```
image: postgres:15-alpine
container_name: postgres-dev
ports: "5432:5432"
volumes:
  - postgres_data:/var/lib/postgresql/data # persistance
  - ./init-multi-db.sql:/docker-entrypoint-initdb.d/init.sql # init au premier démarrage
```

Rôle : Base de données principale. Héberge deux databases :

- `keycloak` : tables internes de Keycloak (utilisateurs, sessions, realm config...)
- `iam_pam_db` : données applicatives (tenants, ressources, demandes, audits...)

Healthcheck : `pg_isready -U postgres` — Keycloak ne démarre qu'après ce check.

Script `init-multi-db.sql` :

```
CREATE DATABASE keycloak;
CREATE DATABASE iam_pam_db;
\c iam_pam_db
CREATE SCHEMA IF NOT EXISTS shared;
CREATE SCHEMA IF NOT EXISTS tenant_bank_a;
CREATE SCHEMA IF NOT EXISTS tenant_bank_b;
CREATE SCHEMA IF NOT EXISTS tenant_fintech_c;
```


4.3 Service `keycloak`

```
image: quay.io/keycloak/keycloak:23.0
container_name: keycloak-dev
depends_on: postgres (service_healthy)
command: start-dev
ports: "8080:8080"
```

Variables d'environnement clés :

Variable	Valeur	Description
KEYCLOAK_ADMIN	admin	Login de la console admin
KEYCLOAK_ADMIN_PASSWORD	admin	Mot de passe console admin
KC_DB	postgres	Driver base de données
KC_DB_URL_HOST	postgres	Hostname PostgreSQL (résolution DNS Docker)
KC_DB_URL_DATABASE	keycloak	Nom de la base
KC_HTTP_ENABLED	true	HTTP autorisé (développement)
KC_HOSTNAME_STRICT	false	Pas de restriction hostname en dev

Rôle : Serveur SSO/OIDC. Gère l'authentification, les tokens JWT, la fédération LDAP.

Configuration manuelle requise (via UI :8080) :

1. Créer le realm `iam-pam-realm`
2. Créer les clients `iam-pam-backend` et `iam-pam-frontend`
3. Créer les rôles de realm (`admin` , `tenant-admin` , `user` , `pam-access` , `auditor`)
4. Configurer User Federation vers OpenLDAP
5. Ajouter le mapper de groupe `groups` dans les claims JWT

4.4 Service `pgadmin`

```
image: dpage/pgadmin4:latest
container_name: pgadmin-dev
ports: "5050:80"
volumes: pgadmin_data
```

Rôle : Interface web PostgreSQL. Accès à `http://localhost:5050` , login `admin@iam-pam.com` / `admin` .

4.5 Service `openldap`

```
image: osixia/openldap:1.5.0
container_name: openldap-dev
ports: "389:389" (LDAP), "636:636" (LDAPS)
volumes:
  - openldap_data:/var/lib/ldap      # données annuaire
  - openldap_config:/etc/ldap/slapd.d # configuration slapd
```

Variables clés :

Variable	Valeur
<code>LDAP_ORGANISATION</code>	IAM PAM Demo
<code>LDAP_DOMAIN</code>	bank-a.local
<code>LDAP_ADMIN_PASSWORD</code>	Admin1234

Rôle : Annuaire LDAP simulant un Active Directory d'entreprise. Keycloak s'y connecte pour la fédération des utilisateurs. Le backend peut aussi s'y connecter directement via `AdService`.

4.6 Service `phpldapadmin`

```
image: osixia/phpldapadmin:0.9.0
ports: "8090:80"
depends_on: openldap
```

Rôle : Interface web pour administrer l'annuaire OpenLDAP. Accès à `http://localhost:8090`.

4.7 Service `redis`

```
image: redis:7-alpine
container_name: redis-dev
command: redis-server --appendonly yes # persistance AOF
ports: "6379:6379"
volumes: redis_data:/data
```

Rôle : Cache en mémoire. Prévu pour les sessions et le rate-limiting. Actuellement déclaré dans l'infrastructure mais non utilisé activement dans le code applicatif (réservé pour évolution).

4.8 Volumes Docker

Volume	Service	Contenu
postgres_data	postgres	Fichiers de données PostgreSQL
pgadmin_data	pgadmin	Configuration et serveurs sauvegardés
redis_data	redis	Journal AOF Redis
openldap_data	openldap	Entrées LDAP (utilisateurs, groupes)
openldap_config	openldap	Configuration du serveur slapd

Tous les volumes sont de type `local` (répertoire sur la machine hôte). La suppression du volume détruit les données.

5. Backend Spring Boot

5.1 Configuration générale

Port : `8081`

Base package : `com.iam.pam`

Profile actif : `local` (surcharge `application-local.properties`)

```
src/main/
├─ java/com/iam/pam/
│   ├── IamPamBackendApplication.java    # Point d'entrée Spring Boot
│   ├── config/                          # Configuration Spring
│   ├── controller/                      # Contrôleurs REST
│   ├── dto/                             # Objets de transfert de données
│   ├── entity/                          # Entités JPA
│   ├── repository/                      # Interfaces Spring Data
│   ├── security/                        # Intercepteur, TenantContext
│   ├── service/                         # Logique métier
│   └─ websocket/                        # Handlers WebSocket
└─ resources/
    ├── application.properties            # Config principale
    ├── application-local.properties      # Secrets locaux (gitignored)
    └─ bastion_key                        # Clé privée SSH pour le bastion
```

5.2 Package `config`

`SecurityConfig.java`

Configure Spring Security comme **OAuth2 Resource Server** stateless.

Décodeur JWT composite :

```
return token -> {  
    String iss = JWTParser.parse(token).getJWTClaimsSet().getIssuer();  
    if ("face-auth".equals(iss)) {  
        return faceDecoder.decode(token); // HMAC-256  
    }  
    return keycloakDecoder.decode(token); // RSA via JWK Set  
};
```

Extraction des rôles :

- Keycloak JWT : `realm_access.roles` → `ROLE_<nom>`
- Face JWT : claim flat `roles` → `ROLE_<nom>`

Matrice d'autorisation des routes :

Chemin	Accès
<code>/api/public/**</code> , <code>/ws/**</code> , <code>/proxy/web/**</code> , <code>/swagger-ui/**</code> , <code>/v3/api-docs/**</code>	Public
<code>/api/admin/tenants/my/**</code>	ROLE_tenant-admin
<code>/api/admin/tenants/**</code>	ROLE_admin, ROLE_tenant-admin
<code>/api/admin/users/**</code>	ROLE_tenant-admin
<code>/api/tenant/services/**</code>	ROLE_tenant-admin, ROLE_user
<code>/api/admin/**</code>	ROLE_admin, ROLE_tenant-admin
<code>/api/user/face/**</code> , <code>/api/user/mfa/**</code>	Authentifié
<code>/api/user/**</code>	ROLE_user, ROLE_tenant-admin
<code>/api/auditor/**</code>	ROLE_auditor, ROLE_tenant-admin
<code>/api/pam/**</code>	Authentifié (contrôle fin par <code>@PreAuthorize</code>)

CORS : Origines autorisées configurées via `cors.allowed-origins` (par défaut `http://localhost:3000`). Credentials autorisés, tous les headers acceptés.

Frame Options désactivées : Requis pour que l'iframe du proxy web fonctionne.

`WebSocketConfig.java`

Enregistre deux handlers WebSocket :

URL	Handler	Protocol
<code>/ws/session/{requestId}</code>	<code>BastionTerminalHandler</code>	Texte (xterm.js)
<code>/ws/rdp/{requestId}</code>	<code>GuacamoleWebSocketHandler</code>	guacamole (sous-protocole WebSocket)

Le sous-protocole `guacamole` est négocié dans le handshake WebSocket (`Sec-WebSocket-Protocol: guacamole`). Le serveur **doit** l'écho pour que le client Guacamole.js accepte la connexion.

`AesEncryptionConverter.java`

JPA `@Converter(autoApply = false)` appliqué manuellement sur les champs `credentialPassword` et `credentialPrivateKey` de l'entité `Resource` .

```
Chiffrement : cleartext → AES-256-GCM → Base64(IV || ciphertext || tag)
Déchiffrement : Base64 → extraire IV (12B) → AES-256-GCM decrypt → cleartext
```

La clé AES est lue depuis `resource.encryption.key` , tronquée/paddée à 32 octets.

`KeycloakConfig.java`

Crée un bean `Keycloak` (client Admin REST) :

- Server URL : `http://localhost:8080`
- Realm : `iam-pam-realm`
- Client : `iam-pam-backend`
- Secret : depuis `application-local.properties`

Utilisé par `KeycloakAdminService` pour gérer les utilisateurs et groupes.

`WebConfig.java`

- Enregistre `TenantInterceptor` sur tous les chemins

- Configure CORS (doublé avec SecurityConfig pour couvrir les erreurs 403 pré-filtre)

SwaggerConfig.java

OpenAPI 3.0 exposé sur :

- /v3/api-docs : JSON de spec
- /swagger-ui.html : UI interactive

5.3 Package `security`

TenantContext.java

```
public class TenantContext {
    private static final ThreadLocal<String> CURRENT_TENANT = new ThreadLocal<>();

    public static void setCurrentTenant(String tenantId) { CURRENT_TENANT.set(tenantId); }
    public static String getCurrentTenant() { return CURRENT_TENANT.get(); }
    public static void clear() { CURRENT_TENANT.remove(); }
}
```

Pourquoi ThreadLocal : Chaque requête HTTP est traitée dans un thread dédié (pool Tomcat). Le ThreadLocal lie la valeur au thread, donc à la requête, sans synchronisation.

TenantInterceptor.java

Implémente `HandlerInterceptor`. Exécuté **avant** chaque requête HTTP authentifiée.

Workflow :

1. Récupère le JWT Spring Security (`Jwt jwt = authentication.getPrincipal()`)
2. Lit le claim `groups` (ex : `["tenant-bank-a"]`)
3. Si absent et non-admin → appel Keycloak Admin API pour récupérer les groupes via `sub`
4. Vérifie que l'utilisateur appartient à un tenant (sauf `admin`)
5. Appelle `TenantContext.setCurrentTenant(tenantId)`
6. Pour les `tenant-admin` : vérifie que le domaine est configuré (sinon 403 `DOMAIN_NOT_CONFIGURED`)
7. Dans `afterCompletion` → `TenantContext.clear()`

Whitelist domaine (exemptés du check domaine configuré) :

- `/api/admin/tenants/my/domain`
- `/api/admin/tenants/my/domain/status`
- `/api/admin/tenants/my`

5.4 Package `entity`

`Tenant.java`

Table : `shared.tenants`

Champ	Type	Description
<code>id</code>	Long (PK)	Identifiant auto-généré
<code>tenantId</code>	String (unique)	Identifiant métier (ex : <code>tenant-bank-a</code>)
<code>tenantName</code>	String	Nom affiché
<code>schemaName</code>	String	Schéma PostgreSQL dédié
<code>domain</code>	String	Domaine LDAP/AD de l'organisation
<code>domainConfigured</code>	Boolean	Flag : domaine configuré
<code>maxUsers</code>	Integer	Quota d'utilisateurs
<code>currentUserCount</code>	Integer	Nombre actuel (géré programmatically)
<code>isActive</code>	Boolean	Tenant actif ou désactivé
<code>adminUsername</code>	String	Username du tenant-admin principal
<code>createdAt</code> , <code>updatedAt</code>	LocalDateTime	Horodatages

Méthodes métier :

- `canAddUser()` : `currentUserCount < maxUsers`
- `incrementUserCount()` / `decrementUserCount()` : avec vérification de contraintes

`Resource.java`

Table : `shared.resources`

Champ	Type	Description
id	Long (PK)	
name	String	Nom lisible (ex : "Serveur DB Prod")
type	Enum ResourceType	SSH , RDP , DATABASE , WEB , API
host	String	Adresse IP ou hostname de la ressource
port	Integer	Port de connexion
description	String	Description libre
tenantId	String	Propriétaire (tenant)
credentialUsername	String	Login cible
credentialPassword	String	Chiffré AES-256-GCM
credentialPrivateKey	String	Clé privée SSH chiffrée
isActive	Boolean	Ressource active
createdAt , updatedAt	LocalDateTime	

`AccessRequest.java`

Table : `shared.access_requests`

Champ	Type	Description
id	Long (PK)	
requesterUsername	String	Username du demandeur
tenantId	String	Tenant concerné
resource	ManyToOne Resource	Ressource demandée
justification	String	Raison de la demande
status	Enum RequestStatus	PENDING , APPROVED , REJECTED , EXPIRED , REVOKED
durationHours	Integer	Durée demandée (nulle = indéfinie)
reviewedBy	String	Username du revieweur

Champ	Type	Description
<code>reviewComment</code>	String	Commentaire de review
<code>requestedAt</code>	LocalDateTime	Date de création
<code>reviewedAt</code>	LocalDateTime	Date de review
<code>expiresAt</code>	LocalDateTime	Date d'expiration calculée

Méthode `isActive()` :

```
return status == APPROVED && (expiresAt == null || expiresAt.isAfter(LocalDateTime.now()));
```

AuditLog.java

Table : `shared.audit_logs`

Champ	Type	Description
<code>id</code>	Long (PK)	
<code>action</code>	Enum <code>AuditAction</code>	Type d'événement
<code>username</code>	String	Acteur
<code>tenantId</code>	String	Tenant concerné
<code>resourceName</code>	String	Ressource impliquée
<code>accessRequestId</code>	Long	Référence à la demande
<code>details</code>	String	Description détaillée
<code>ipAddress</code>	String	IP du client
<code>result</code>	Enum <code>AuditResult</code>	<code>SUCCESS</code> , <code>FAILURE</code>
<code>timestamp</code>	LocalDateTime	Immuable, indexé

Enum `AuditAction` :

`USER_LOGIN` , `USER_LOGOUT` , `ACCESS_REQUESTED` , `ACCESS_APPROVED` , `ACCESS_REJECTED` ,
`ACCESS_REVOKED` , `ACCESS_EXPIRED` , `RESOURCE_CREATED` , `RESOURCE_UPDATED` ,
`RESOURCE_DELETED` , `SESSION_STARTED` , `SESSION_ENDED` , `COMMAND_EXECUTED` ,
`PERMISSION_DENIED`

UserMfaEnrollment.java

Table : `shared.user_mfa_enrollment`

Champ	Type	Description
<code>tenantId + username</code>	Unique pair	Identifiant logique
<code>method</code>	Enum <code>MfaMethod</code>	TOTP , EMAIL , SMS , WHATSAPP
<code>secret</code>	String	Secret TOTP (Base32)
<code>contactEmail</code>	String	E-mail pour OTP
<code>phoneNumber</code>	String	Téléphone pour SMS/WhatsApp
<code>isActive</code>	Boolean	Enrollment confirmé
<code>enrolledAt</code>	LocalDateTime	Date d'activation
<code>lastVerifiedAt</code>	LocalDateTime	Dernière vérification

TenantMfaConfig.java

Table : `shared.tenant_mfa_config` (une ligne par tenant)

Flags booléens : `totpEnabled` , `emailOtpEnabled` , `smsOtpEnabled` ,
`whatsappOtpEnabled` , `mfaRequired`

UserFaceDescriptor.java

Table : `shared.user_face_descriptors`

Champ	Description
<code>tenantId + username</code>	Identifiant logique
<code>descriptorJson</code>	Vecteur Float32 de 128 dimensions, sérialisé en JSON
<code>isActive</code>	Enrollment actif
<code>createdAt</code>	Date d'enrollment

TenantAdConfig.java

Table : `shared.tenant_ad_configs`

Configuration de connexion LDAP/AD par tenant :

Champ	Description
serverUrl	URL du serveur LDAP (ex : ldap://openldap)
port	Port (389 ou 636)
useSsl	TLS activé
bindDn	DN de connexion (compte de service)
bindPassword	Mot de passe du compte de service
userSearchBase	Branche de recherche (ex : ou=users,dc=bank-a,dc=local)
userSearchFilter	Filtre LDAP (ex : (objectClass=inetOrgPerson))
usernameAttribute , emailAttribute ...	Mapping des attributs LDAP

TenantService.java (entité)

Table : shared.tenant_services

Abonnement d'un tenant à un service (IAM , PAM , MFA , SSO , AUDIT , REPORTING).

Champs : tenantId , serviceType , isActive , subscribedAt , expiresAt .

Méthode isExpired() : expiresAt != null && expiresAt.isBefore(now) .

5.5 Package repository

Tous les repositories étendent JpaRepository<Entity, Long> et bénéficient automatiquement du CRUD standard.

TenantRepository

```
Optional<Tenant> findByTenantId(String tenantId);
Optional<Tenant> findByTenantIdForUpdate(String tenantId); // @Lock(PESSIMISTIC_WRITE)
boolean existsByTenantId(String tenantId);
Optional<Tenant> findBySchemaName(String schemaName);
boolean existsByDomain(String domain);
List<Tenant> findByIsActiveTrue();
```

ResourceRepository

```
List<Resource> findByTenantId(String tenantId);
List<Resource> findByTenantIdAndIsActiveTrue(String tenantId);
Optional<Resource> findByNameAndTenantId(String name, String tenantId);
List<Resource> findByTypeAndTenantId(ResourceType type, String tenantId);
boolean existsByNameAndTenantId(String name, String tenantId);
```

AccessRequestRepository (+ JpaSpecificationExecutor)

```
List<AccessRequest> findByRequesterUsernameOrderByRequestedAtDesc(String username);
List<AccessRequest> findByRequesterUsernameAndStatusOrderByRequestedAtDesc(String username, RequestStatus status);
List<AccessRequest> findByTenantIdOrderByRequestedAtDesc(String tenantId);
List<AccessRequest> findByTenantIdAndStatusOrderByRequestedAtDesc(String tenantId, RequestStatus status);

@Query("SELECT ar FROM AccessRequest ar WHERE ar.status = :status AND ar.expiresAt IS NOT NULL")
List<AccessRequest> findExpiredRequests(LocalDateTime now, RequestStatus status);

@Query("SELECT COUNT(ar) FROM AccessRequest ar WHERE ar.tenantId = :tenantId AND ar.status = :status")
long countActiveSessions(String tenantId, RequestStatus status, LocalDateTime now);

@Query("SELECT ar FROM AccessRequest ar JOIN FETCH ar.resource WHERE ar.id = :id")
Optional<AccessRequest> findByIdWithResource(Long id);

@Query("SELECT ar FROM AccessRequest ar JOIN FETCH ar.resource WHERE ar.id IN :ids")
List<AccessRequest> findByIdsWithResource(List<Long> ids);
```

AuditLogRepository (+ JpaSpecificationExecutor)

```
Page<AuditLog> findByTenantIdOrderByTimestampDesc(String tenantId, Pageable pageable);
List<AuditLog> findByUsernameAndTenantIdOrderByTimestampDesc(String username, String tenantId);
List<AuditLog> findByTenantIdAndActionAndTimestampAfterOrderByTimestamp(String tenantId, AuditAction action, LocalDateTime after);
List<AuditLog> findByTenantIdAndActionAndTimestampBetweenOrderByTimestampDesc(String tenantId, AuditAction action, LocalDateTime start, LocalDateTime end);

@Query("SELECT a.resourceName, COUNT(a) FROM AuditLog a WHERE a.tenantId = :tenantId AND a.action = :action")
List<Object[]> countSessionsByResource(String tenantId, AuditAction action);
```

UserMfaEnrollmentRepository

```
Optional<UserMfaEnrollment> findByTenantIdAndUsername(String tenantId, String username);

@Modifying
@Query("DELETE FROM UserMfaEnrollment e WHERE e.tenantId = :tenantId AND e.username = :username")
void deleteByTenantIdAndUsername(String tenantId, String username);
```

FaceDescriptorRepository

```
Optional<UserFaceDescriptor> findByTenantIdAndUsernameAndIsActiveTrue(String tenantId, String username);
List<UserFaceDescriptor> findAllByUsernameAndIsActiveTrue(String username); // cross-tenant
@Modifying void deleteByTenantIdAndUsername(String tenantId, String username);
```

5.6 Package `controller`

Tous les contrôleurs renvoient `ResponseEntity<ApiResponse<T>>`.

ApiResponse<T> :

```
{
  "success": true,
  "message": "Logs retrieved",
  "data": { ... },
  "timestamp": "2026-05-08T14:30:00"
}
```

TenantController — `/api/admin/tenants`

Méthode	URL	Rôle	Description
POST	/	admin	Créer un tenant avec ses services
GET	/	admin, tenant-admin	Lister tous les tenants
GET	/tenantId	admin, tenant-admin	Détail d'un tenant
GET	/my	tenant-admin	Tenant courant de l'admin connecté

Méthode	URL	Rôle	Description
PUT	/tenantId	admin	Mettre à jour un tenant
DELETE	/tenantId	admin	Désactiver un tenant
POST	/my/domain	tenant-admin	Configurer le domaine AD
GET	/my/domain/status	tenant-admin	Statut de la configuration domaine

ResourceController — /api/pam/resources

Méthode	URL	Rôles	Description
GET	/	user, pam-access, tenant-admin	Lister les ressources du tenant
GET	/id	user, pam-access, tenant-admin	Détail d'une ressource
POST	/	tenant-admin	Créer une ressource
PUT	/id	tenant-admin	Mettre à jour une ressource
DELETE	/id	tenant-admin	Désactiver (soft delete)

AccessRequestController — /api/pam/requests

Méthode	URL	Rôles	Description
GET	/	tenant-admin	Toutes les demandes du tenant
GET	/pending	tenant-admin	Demandes en attente
GET	/mine	user, pam-access, tenant-admin	Mes demandes
GET	/active	user, pam-access, tenant-admin	Sessions actives
POST	/	user, pam-access, tenant-admin	Créer une demande
PUT	/id/review	tenant-admin	Approuver / Rejeter
PUT	/id/revoke	tenant-admin	Révoquer

Méthode	URL	Rôles	Description
PUT	<code>/{{id}}/terminate</code>	user	Terminer sa propre session

AuditController — `/api/auditor`

Méthode	URL	Description
GET	<code>/logs?page=&size=&username=&action=&dateFrom=&dateTo=</code>	Logs paginés avec filtres (JPA Specification)
GET	<code>/stats</code>	Statistiques dashboard (sessions actives, sessions/jour J-7)
GET	<code>/resource-usage</code>	Nombre de sessions par ressource
GET	<code>/sessions-by-day?date=YYYY-MM-DD</code>	Détail des sessions d'un jour
GET	<code>/logs/user/{username}</code>	Journal d'un utilisateur
GET	<code>/logs/export</code>	Export CSV (fichier audit-logs.csv)

MfaController — `/api/user/mfa`

Méthode	URL	Description
GET	<code>/tenant-config</code>	Configuration MFA du tenant
PUT	<code>/tenant-config</code>	Mettre à jour (tenant-admin)
GET	<code>/status</code>	Statut d'enrollment de l'utilisateur connecté
POST	<code>/enroll/init</code>	Démarrer l'enrollment
POST	<code>/enroll/confirm</code>	Confirmer avec le code reçu
DELETE	<code>/enroll</code>	Supprimer l'enrollment

Méthode	URL	Description
POST	/send-otp	Renvoyer le code OTP
POST	/verify	Vérifier le code post-login

FaceDescriptorController — /api/user/face

Méthode	URL	Description
POST	/enroll	Enregistrer le descripteur facial
POST	/verify	Vérifier → retourner {match, distance}
GET	/status	Statut d'enrollment
DELETE	/enroll	Supprimer l'enrollment facial

AdController — /api/admin/ad

Méthode	URL	Description
POST	/config	Sauvegarder la config AD/LDAP
GET	/config	Récupérer la config (mot de passe masqué)
GET	/config/status	Vérifier si configuré
POST	/test	Tester la connexion LDAP
GET	/users/search?query=	Rechercher dans l'AD
POST	/users/import	Importer un utilisateur AD vers Keycloak
POST	/users/create	Créer un utilisateur dans l'AD

5.7 Package `service`

BastionService

Gère les sessions SSH via le bastion.

Attributs :

- `sshClient` : instance Apache SSHD démarrée au démarrage (`@PostConstruct` implicite)
- `activeSessions` : `ConcurrentHashMap<Long requestId, ActiveSession>`

Méthodes :

`openSession(accessRequestId, wsSession) :`

1. Charge l' `AccessRequest` avec `JOIN FETCH` sur la ressource
2. Vérifie `request.isActive()`
3. Connecte Apache SSHD au bastion (`bastionHost:22` , auth par clé privée RSA)
4. Ouvre un canal shell PTY (`xterm-256color`)
5. Lie les pipes I/O : `[stdin browser] → toShell` et `shellOutput → [stdout browser]`
6. Exécute `buildSshCommand() : sshpass -p 'pass' ssh -o StrictHostKeyChecking=no -p port user@host` (ou sans `sshpass` si pas de mot de passe)
7. Démarre un thread démon `bastion-relay-{id}` qui relaie le STDOUT vers le WebSocket
8. Logue `SESSION_STARTED` dans l'audit

`sendInput(accessRequestId, input) :`

- Écrit dans `session.toShell()`
- Logue `COMMAND_EXECUTED` pour chaque keystroke non vide (hors contrôles)

`closeSession(accessRequestId) :`

- Ferme le shell et la session SSH
- Logue `SESSION_ENDED`
- Nettoie la `activeSessions` map

`loadKeyPair()` : charge la clé privée depuis `classpath:bastion_key` via Apache SSHD `FileKeyPairProvider` .

Nettoyage : `@PreDestroy` ferme toutes les sessions actives à l'arrêt du serveur.

GuacamoleService

Gère les sessions RDP via Apache Guacamole.

Attributs :

- `activeSessions` : `ConcurrentHashMap<Long requestId, ActiveRdpSession>`

Méthodes :

`openSession(accessRequestId, wsSession) :`

1. Charge l' `AccessRequest`
2. Construit la `GuacamoleConfiguration` (protocole RDP, host, port, username, password, résolution 1280×720, 16bpp, `ignore-cert=true`)
3. Ouvre un `InetGuacamoleSocket(guacdHost, guacdPort) → ConfiguredGuacamoleSocket` (handshake guacd)
4. Crée un `SimpleGuacamoleTunnel`
5. Démarre un thread démon `guac-relay-{id}` qui lit les instructions Guacamole et

les pousse dans le WebSocket

6. Logue SESSION_STARTED

`sendInstruction(accessRequestId, instruction)` :

- Acquiert le writer du tunnel guacd, écrit l'instruction Guacamole (texte), relâche le writer

`closeSession(accessRequestId)` :

- Ferme le tunnel Guacamole

- Logue SESSION_ENDED

AccessRequestService

```
createRequest(dto, username)
    → vérifie que la ressource appartient au tenant courant
    → crée avec status PENDING
    → log ACCESS_REQUESTED

reviewRequest(id, dto, reviewerUsername)
    → vérifie status == PENDING
    → change status en APPROVED/REJECTED
    → si APPROVED et durationHours non null → calcule expiresAt = now + hours
    → log ACCESS_APPROVED ou ACCESS_REJECTED

revokeRequest(id, adminUsername)
    → change status en REVOKED
    → log ACCESS_REVOKED

revokeOwnSession(id, username)
    → vérifie que c'est bien la session de l'utilisateur
    → log ACCESS_REVOKED

@Scheduled(fixedDelay = 60000) // Toutes les 60 secondes
expireRequests()
    → findExpiredRequests(now, APPROVED) → status = EXPIRED
```

MfaService

Gère l'enrollment et la vérification MFA. Les OTP sont stockés en mémoire :

```
Map<String, long[]> otpStore; // key = "tenantId:username" → [hashCode(code), expireEpoch]
Map<String, String> pendingTotpSecrets; // key = "tenantId:username" → secret TOTP en cours
```

TOTP (RFC 6238) :

```
T = floor(now_seconds / 30)
code = HOTP(secret, T) avec HOTP = HMAC-SHA1(base32decode(secret), T) → troncature dynamique
```

Accepte T-1, T, T+1 (dérive d'horloge $\pm 30s$).

OTP numérique : `SecureRandom.nextInt(1_000_000)` → formaté sur 6 chiffres. TTL : 300 secondes.

Sécurité OTP : stockage du `hashCode(code)` et non du code en clair en mémoire.

FaceTokenService

Génère des JWT HS256 pour les utilisateurs authentifiés par Face ID :

```
{
  "sub": "alice",
  "iss": "face-auth",
  "preferred_username": "alice",
  "roles": ["user"],
  "tenantId": "tenant-bank-a",
  "groups": ["tenant-bank-a"],
  "iat": 1746700000,
  "exp": 1746786400
}
```

Durée par défaut : 24h (configurable `face.jwt.expiry-hours`).

FaceDescriptorService

`enrollFace(username, descriptor[])` :

1. Désactive l'enrollment précédent (`isActive = false`)
2. Crée une nouvelle entrée avec le JSON du descripteur

`verifyFace(username, descriptor[])` :

1. Charge le descripteur stocké
2. Calcule la **distance euclidienne** entre les deux vecteurs `Float32`
3. Retourne `{match: distance < 0.5, distance}`

`getStatus(username)` : vérifie si un enrollment actif existe.

AuditService

`log(action, username, tenantId, resourceName, requestId, details) :`

Crée et persiste un `AuditLog`. Ne lève jamais d'exception (erreurs catchées silencieusement pour ne pas bloquer le flux principal).

`getFilteredLogs(page, size, username, action, dateFrom, dateTo) :`

Utilise JPA `Specification` (Criteria API) :

```
Specification<AuditLog> spec = (root, query, cb) -> {
    List<Predicate> predicates = new ArrayList<>();
    predicates.add(cb.equal(root.get("tenantId"), tenantId));
    if (username != null) predicates.add(cb.like(cb.lower(root.get("username")), "%" + username));
    if (action != null) predicates.add(cb.equal(root.get("action"), action));
    if (fromDate != null) predicates.add(cb.greaterThanOrEqualTo(root.get("timestamp"), fromDate));
    if (toDate != null) predicates.add(cb.lessThanOrEqualTo(root.get("timestamp"), toDate));
    return cb.and(predicates.toArray(new Predicate[0]));
};
```

Pourquoi Specification ? Les paramètres nuls JPQL (`?1 IS NULL OR col = ?1`) causent une erreur PostgreSQL sur les colonnes typées (enum, timestamp). Avec `Specification`, on n'ajoute le prédicat que si le paramètre est non nul.

`getStats() :`

- `activeSessions` : `countActiveSessions(tenantId, APPROVED, now)`
- `sessionsByDay` : logs `SESSION_STARTED` des 7 derniers jours, groupés par date

`getResourceUsage() :`

- `countSessionsByResource(tenantId, SESSION_STARTED) → [(resourceName, count)]`
- Enrichi avec le `resourceType` via `ResourceRepository.findByTenantId()`

`getSessionsForDay(dateStr) :`

1. Filtre les `SESSION_STARTED` du jour
2. Extrait les `accessRequestId` des détails
3. Batch-load des `AccessRequest` avec `JOIN FETCH resource` (évite N+1)
4. Retourne les détails complets (user, resource, type, start, duration, status)

KeycloakAdminService

Encapsule les appels à l'API Admin REST Keycloak :

- `searchDirectoryUsers(query)` : recherche parmi les utilisateurs fédérés LDAP
- `listDirectoryUsers(first, max)` : pagination de la liste
- `importUserToTenant(tenantId, username, roles)` : ajoute l'utilisateur au groupe Keycloak du tenant, assigne les rôles
- `removeUserFromTenant(tenantId, username)` : retire du groupe

- `createTenantGroup(tenantId)` : crée le groupe Keycloak
 - `getUserFirstTenantGroup(userId)` : fallback si le claim `groups` est absent du JWT
-

AdService

Gère la configuration et les opérations sur l'Active Directory/LDAP :

- `saveConfig(SaveRequest)` : persiste la config dans `TenantAdConfig`
 - `testConnection()` : tente un bind LDAP avec les credentials configurés
 - `searchUsers(query)` : `LdapTemplate.search()` avec filtre dynamique
 - `importAdUser(ImportRequest)` : crée l'utilisateur dans Keycloak via `KeycloakAdminService.importUserToTenant()`
 - `createAdUser(CreateRequest)` : crée dans l'AD via `LdapTemplate.create()` , optionnellement importe dans Keycloak
-

NotificationService

`sendEmailOtp(email, code)` :

- Si `mfa.email.enabled=true` et SMTP configuré → `JavaMailSender.send()`
- Sinon → log `[MFA-EMAIL] code=XXXXXX` en console (mode développement sans SMTP)

`sendSmsOtp(phone, code)` :

- Si `TWILIO_ACCOUNT_SID` configuré → appel API Twilio REST (`Message.creator().create()`)
- Sinon → log console

`sendWhatsAppOtp(phone, code)` :

- Même que SMS, avec `from = whatsapp:+14155238886`
-

TenantManagementService

`createTenant(CreateRequest)` :

1. Vérifie unicité du `tenantId`
2. Génère `schemaName = "tenant_" + tenantId.replace("-", "_")`
3. Crée le groupe Keycloak via `KeycloakAdminService.createTenantGroup()`
4. Persiste le `Tenant`
5. Crée les entrées `TenantService` pour les services demandés

`updateTenant(tenantId, UpdateRequest)` :

- Met à jour `tenantName`, `maxUsers`, `isActive`
 - Synchronise les `TenantService` (active/désactive les services)
-

TenantServiceChecker

```
public void requireService(String tenantId, ServiceType serviceType) {
    if (!isServiceAvailable(tenantId, serviceType)) {
        throw new ServiceNotSubscribedException("Service " + serviceType + " not subscribed fo
    }
}
```

Appelé dans les services PAM pour vérifier que le tenant a souscrit au module PAM avant d'autoriser les opérations.

WebProxyService

Proxy HTTP transparent pour les ressources de type `WEB`.

- Reçoit une requête sur `/proxy/web/{requestId}/**`
 - Vérifie la validité de la session (`AccessRequest.isActive()`)
 - Forward la requête HTTP vers `http://{resource.host}:{resource.port}/{path}`
 - Retourne la réponse au navigateur (headers, body)
 - Gère les redirections et la réécriture des liens relatifs
-

DbTunnelService

Crée un tunnel TCP local pour les ressources de type `DATABASE`.

- Ouvre un `ServerSocket` local sur un port aléatoire
 - Établit une connexion TCP vers `resource.host:resource.port`
 - Relais les données bidirectionnellement
 - Retourne l'URL de connexion (`jdbc:postgresql://127.0.0.1:{localPort}/...`)
-

5.8 Package `websocket`

BastionTerminalHandler

Étend `TextWebSocketHandler`. Enregistré sur `/ws/session/{requestId}`.

```

afterConnectionEstablished()
    1. Extraire ?token= de l'URI
    2. jwtDecoder.decode(token) ← lève JwtException si invalide
    3. Extraire requestId du path
    4. sessionRequestMap.put(wsSession.getId(), requestId)
    5. bastionService.openSession(requestId, wsSession)

handleTextMessage()
    1. Récupérer requestId depuis sessionRequestMap
    2. Si payload.startsWith("RESIZE:") → ignoré (réservé futur)
    3. bastionService.sendInput(requestId, payload)

afterConnectionClosed()
    1. bastionService.closeSession(requestId)

handleTransportError()
    1. bastionService.closeSession(requestId)

```

GuacamoleWebSocketHandler

Étend `TextWebSocketHandler`. Enregistré sur `/ws/rdp/{requestId}`.

Identique à `BastionTerminalHandler` dans la gestion JWT et du path, mais :

- `afterConnectionEstablished` : démarre guacd setup dans un **thread démon séparé** (le `ConfiguredGuacamoleSocket` bloque ~800ms pendant le handshake guacd, il ne doit pas bloquer le thread Tomcat)
- `handleTextMessage` : `guacamoleService.sendInstruction(requestId, payload)` (instructions Guacamole)

5.9 DTOs

ApiResponse<T>

```

{ success: boolean, message: String, data: T, timestamp: LocalDateTime }
static ApiResponse.success(data, message)
static ApiResponse.error(message)

```

TenantDTO

- `CreateRequest` : `tenantId`, `tenantName`, `maxUsers`, `services[]`
- `UpdateRequest` : `tenantName`, `maxUsers`, `services[]`, `isActive`
- `DomainConfigRequest` : `domain`
- `Response` : tous les champs Tenant + liste des services avec statut

ResourceDTO

- Request : name, type, host, port, description, credentialUsername, credentialPassword, credentialPrivateKey
- Response : tous les champs sauf les credentials en clair. hasPassword et hasPrivateKey sont des booléens indiquant la présence.

AccessRequestDTO

- Request : resourceId, justification, durationHours
- ReviewRequest : status (APPROVED/REJECTED), comment
- Response : tous les champs + resourceName, resourceType, isActive

MfaDTO

- TenantConfigRequest/Response : flags booléens + mfaRequired
- InitEnrollRequest : method, contactEmail?, phoneNumber?
- InitEnrollResponse : method, totpSecret?, totpQrUri?, message
- ConfirmEnrollRequest : code
- VerifyRequest/Response : code, success, message
- SendOtpResponse : message
- StatusResponse : enrolled, method?, enrolledAt?

FaceDescriptorDTO

- EnrollRequest / VerifyRequest : descriptor[] (float array)
- VerifyResponse : match (boolean), distance (float)
- StatusResponse : enrolled, enrolledAt?

AuditLogDTO.Response

Tous les champs de AuditLog avec les enums en String.

AdConfigDTO

- SaveRequest : tous les champs de TenantAdConfig avec validation (@NotBlank, @Min)
 - Response : idem avec bindPassword masqué (***)
 - AdUser : username, email, firstName, lastName, distinguishedName
-

6. Frontend Angular

6.1 Structure du projet

```
src/app/
├─ app.module.ts           # Module racine
├─ app-routing.module.ts   # Routes globales
├─ core/
│   ├── guards/
│   │   └─ auth.guard.ts   # Protection des routes
│   ├── interceptors/
│   │   └─ auth.interceptor.ts # Injection du JWT
│   ├── models/            # Interfaces TypeScript
│   └─ services/           # Services HTTP
├─ pages/
│   ├── admin/             # Vues super-admin
│   ├── tenant-admin/      # Vues tenant-admin
│   ├── user/              # Vues utilisateur
│   ├── auditor/           # Vues auditeur
│   ├── profile/           # Profil + MFA + Face
│   ├── login/             # Page de connexion
│   └─ mfa-verify/         # Vérification MFA
└─ shared/
    └─ layout/             # Layout principal (navbar, sidebar)
```

6.2 Configuration

environment.ts (développement) :

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8081/api',
  keycloak: {
    url: 'http://localhost:8080',
    realm: 'iam-pam-realm',
    clientId: 'iam-pam-frontend'
  }
};
```

6.3 **app.module.ts**

Imports principaux :

- **KeycloakAngularModule** : intégration Keycloak
- **BrowserAnimationsModule** : animations Angular Material
- **ReactiveFormsModule** / **FormsModule** : formulaires

- Modules Angular Material (toolbar, table, sidenav, form, dialog, icon, button, paginator, snackbar...)

Providers :

- `APP_INITIALIZER` : initialise Keycloak avec `onLoad: 'check-sso'` (vérifie la session sans forcer le login)
 - `HTTP_INTERCEPTORS` : `AuthInterceptor` pour injecter le Bearer token
-

6.4 AuthGuard

Étend `KeycloakAuthGuard`.

`isAccessAllowed(route, state)` :

1. Si non authentifié Keycloak ET non face-auth → rediriger vers `/login`
2. Si authentifié mais mauvais rôle → rediriger vers la page d'accueil du rôle actuel
3. Si rôle correct → laisser passer

Mapping rôle → page d'accueil :

- `admin` → `/admin/dashboard`
 - `tenant-admin` → `/tenant-admin/dashboard`
 - `auditor` → `/auditor/logs`
 - `pam-access` → `/tenant-admin/resources`
 - `user / défaut` → `/user/my-requests`
-

6.5 AuthInterceptor

Implémente `HttpInterceptor`.

```
intercept(req, next) {
  if (isPublicUrl(req.url)) return next.handle(req);

  const token = isFaceAuth
    ? faceAuthService.getToken()
    : await keycloakService.getToken(); // rafraîchit si expiré

  return next.handle(req.clone({
    headers: req.headers.set('Authorization', `Bearer ${token}`)
  }));
}
```

6.6 AuthService

Point d'entrée unique pour l'identité de l'utilisateur connecté.

```

get isFaceAuth(): boolean {
    return !this.keycloak.isLoggedIn() && this.faceAuth.isAuthenticated();
}

get username(): string {
    if (isFaceAuth) return faceAuth.getUsername();
    return jwt.preferred_username;
}

get roles(): string[] {
    if (isFaceAuth) return faceAuth.getRoles();
    return keycloak.getUserRoles(true);
}

get tenantId(): string | null {
    if (isFaceAuth) return faceAuth.getTenantId();
    return jwt.groups[0];
}

async getToken(): Promise<string> {
    if (isFaceAuth) return faceAuth.getToken();
    return keycloak.getToken(); // async – rafraîchit si nécessaire
}

```

6.7 FaceAuthService

Stockage en `localStorage` :

- `iam_face_token` : JWT signé HS256 (24h)
- `iam_face_profile` : { username, roles, tenantId, loginTime } en JSON

Méthodes clés :

- `getToken()` : lit `localStorage.getItem(FACE_TOKEN_KEY)`
 - `isAuthenticated()` : parse le JWT, vérifie `exp > now`
 - `hasExpiredToken()` : token présent mais expiré (candidat au refresh silencieux)
 - `refreshToken()` : `POST /api/public/face/refresh { token }` → nouveau JWT
 - `saveSession(resp)` : persiste token + profil
 - `clearToken()` : efface `localStorage`
-

6.8 Services HTTP

ResourceService — /api/pam/resources

```
getAll(): Observable<ResourceResponse[]>
getById(id): Observable<ResourceResponse>
create(payload): Observable<ResourceResponse>
update(id, payload): Observable<ResourceResponse>
delete(id): Observable<void>
```

AccessRequestService — /api/pam/requests

```
getAll(): Observable<AccessRequestResponse[]>           // tenant-admin
getPending(): Observable<AccessRequestResponse[]>       // tenant-admin
getMine(): Observable<AccessRequestResponse[]>          // user
getActive(): Observable<AccessRequestResponse[]>        // user
create(payload): Observable<AccessRequestResponse>
review(id, payload): Observable<AccessRequestResponse>
revoke(id): Observable<AccessRequestResponse>
terminate(id): Observable<AccessRequestResponse>
```

TenantService — /api/admin/tenants

```
getAll(): Observable<TenantResponse[]>
getById(tenantId): Observable<TenantResponse>
create(payload): Observable<TenantResponse>
update(tenantId, payload): Observable<TenantResponse>
deactivate(tenantId): Observable<void>
activate(tenantId): Observable<TenantResponse>
getMyTenant(): Observable<TenantResponse>               // GET /my
configureDomain(payload): Observable<TenantResponse>    // POST /my/domain
isDomainConfigured(): Observable<boolean>
```

MfaService — /api/user/mfa

```
getTenantConfig(): Observable<TenantMfaConfig>
updateTenantConfig(cfg): Observable<TenantMfaConfig>
getStatus(): Observable<MfaStatus>
initEnroll(method, contactEmail?, phoneNumber?): Observable<InitEnrollResponse>
confirmEnroll(code): Observable<string>
removeEnrollment(): Observable<string>
sendOtp(): Observable<{message}>
verify(code): Observable<VerifyResponse>
```

FaceService — /api/user/face + face-api.js

```
loadModels(): Promise<void> // Charge les modèles TensorFlow depuis CDN
detectDescriptor(videoEl): Promise<Float32Array | null> // Détecte + calcule le descripteur
enrollFace(descriptor): Observable<void>
verifyFace(descriptor): Observable<{match, distance}>
getStatus(): Observable<{enrolled, enrolledAt?}>
removeEnrollment(): Observable<void>
```

AuditService — /api/auditor

```
getLogs(page, size, username?, action?, dateFrom?, dateTo?): Observable<PageResponse<AuditLogR>
getUserLogs(username): Observable<AuditLogResponse[]>
getStats(): Observable<{activeSessions, sessionsByDay: {date, count}[]}>
getResourceUsage(): Observable<{resourceName, resourceType, sessionCount}[]>
getSessionsByDay(date): Observable<any[]>
exportLogs(): Observable<Blob>
```

6.9 Pages et composants

LoginComponent

Route : /login (publique)

- Bouton "Se connecter avec Keycloak" → `keycloak.login()`
- Section "Connexion par Face ID" :
- Saisie du username
- Chargement des modèles face-api.js
- Accès à la caméra (`getUserMedia`)
- Capture du descripteur facial

- `POST /api/public/face/login { username, descriptor[] }`
 - Si match → `faceAuth.saveSession(resp)` → redirect selon rôle
 - Bouton "MFA Requis" → redirect `/mfa-verify`
-

MfaVerifyComponent

Route : `/mfa-verify` (semi-publique, hors layout)

- Chargée après login Keycloak si `mfaRequired = true`
 - `GET /api/user/mfa/status` → déterminer la méthode d'enrollment
 - `POST /api/user/mfa/send-otp` → envoyer OTP si EMAIL/SMS/WhatsApp
 - Saisie du code
 - `POST /api/user/mfa/verify { code }` → si succès, redirect vers page d'accueil
-

LayoutComponent

Enveloppe toutes les pages protégées. Structure :

```
MatToolbar (header)
- Logo + Nom de l'application
- Nom/rôle de l'utilisateur connecté
- Bouton Déconnexion
MatSidenav (navigation latérale, role-based)
- admin      : Dashboard, Tenants
- tenant-admin: Dashboard, Utilisateurs, Ressources, Demandes, Audit, Config AD, Config MFA
- auditor    : Journaux d'audit
- user       : Mes demandes, Mes sessions
- Tous      : Profil
RouterOutlet (contenu principal)
```

AdminDashboardComponent

Route : `/admin/dashboard` (admin)

- Statistiques globales de la plateforme
 - Liste des tenants avec statut actif/inactif
 - Liens vers la gestion des tenants
-

TenantListComponent

Route : `/admin/tenants` (admin)

- Tableau de tous les tenants
 - Formulaire de création : `tenantId` , `tenantName` , `maxUsers` , checkboxes services
 - Actions : modifier, activer/désactiver
 - Dialogue de détail avec les services souscrits
-

DomainSetupComponent

Route : `/setup` (tenant-admin)

- Affiché au premier login d'un tenant-admin si `domainConfigured = false`
 - Formulaire de configuration domaine AD
 - `POST /api/admin/tenants/my/domain { domain }`
 - Redirige vers le dashboard après succès
-

DashboardComponent (tenant-admin)

Route : `/tenant-admin/dashboard`

KPI Cards avec overlays au clic :

- **Utilisateurs** : `currentUserCount / maxUsers` + barre de progression
- **Ressources** : liste par type (SSH/RDP/...)
- **Demandes en attente** : liste des demandes récentes avec statuts
- **Services actifs** : liste des services souscrits

Graphique en barres (sessions/jour) :

- 7 barres (J-6 à aujourd'hui)
- Hauteur proportionnelle au nombre de sessions
- Clic sur une barre → chargement du détail (`GET /api/auditor/sessions-by-day?date=YYYY-MM-DD`)
- Panneau expandable avec la liste des sessions : utilisateur, ressource, type, heure, durée, statut

Graphique donut (utilisation des ressources) :

- SVG pur (sans librairie) : cercles superposés avec `stroke-dasharray` / `stroke-dashoffset`
- Rotation `-90°` pour démarrer à 12h
- Une tranche par ressource, couleur par type

- Clic sur une tranche → filtre la liste de sessions dessous
- Légende cliquable

Appels API du composant :

```
ngOnInit() {  
  tenantService.getMyTenant()  
  requestService.getPending()  
  resourceService.getAll()  
  auditService.getStats()  
  auditService.getResourceUsage()  
}  
selectDay(date) → auditService.getSessionsByDay(date)
```

UsersComponent

Route : /tenant-admin/users

- Tableau des utilisateurs Keycloak du tenant
- Onglets : Keycloak / AD
- Recherche dans l'AD : GET /api/admin/users/directory/search?query=
- Import d'un utilisateur AD : POST /api/admin/users/import
- Suppression : DELETE /api/admin/users/{username}
- Attribution de rôles

ResourcesComponent

Route : /tenant-admin/resources

- Tableau des ressources PAM
 - Colonnes : nom, type (icône colorée), host:port, statut
 - Formulaire création/édition :
 - Type : SSH / RDP / DATABASE / WEB / API
 - Champs host, port, description
 - Credentials : username, password (masqué), clé privée (textarea)
 - Soft delete : DELETE /api/pam/resources/{id} → isActive = false
 - Indicateur hasPassword / hasPrivateKey (les credentials ne sont jamais renvoyés en clair)
-

RequestsComponent

Route : `/tenant-admin/requests`

- Tableau des demandes d'accès du tenant
 - Filtres : statut, utilisateur, ressource
 - Actions : Approuver / Rejeter (dialogue avec commentaire) / Révoquer
 - Badges colorés par statut (PENDING=orange, APPROVED=vert, REJECTED=rouge, REVOKED=gris, EXPIRED=jaune)
-

AuditComponent

Route : `/tenant-admin/audit`

Tableau des journaux d'audit du tenant :

- Filtres : username, action, dateFrom, dateTo
 - Pagination côté serveur (page, size)
 - `AuditLogResponse` : action, username, resource, details, ip, résultat, timestamp
 - Export CSV
-

AdConfigComponent

Route : `/tenant-admin/ad-config`

- Formulaire de configuration AD/LDAP (serverUrl, port, SSL, bindDn, bindPassword, baseDN, attributs)
 - Bouton "Tester la connexion" : `POST /api/admin/ad/test`
 - Section recherche : saisie → `GET /api/admin/ad/users/search?query=`
 - Bouton "Importer" sur chaque résultat : `POST /api/admin/ad/users/import`
-

MfaConfigComponent

Route : `/tenant-admin/mfa-config`

- Toggles pour activer/désactiver chaque méthode MFA (TOTP, EMAIL, SMS, WhatsApp)
 - Toggle "MFA Requis" (force tous les utilisateurs)
 - `GET /api/user/mfa/tenant-config` → `PUT /api/user/mfa/tenant-config`
-

AuditorLogsComponent

Route : `/auditor/logs`

Interface avancée pour les auditeurs :

- Filtres : username, action (dropdown `AuditAction`), dateFrom, dateTo
 - Pagination avec `mat-paginator` (liaison `[pageIndex]="currentPage"` pour persister la page à travers les rechargements)
 - Export CSV : `GET /api/auditor/logs/export` → `Blob` → téléchargement navigateur
 - Reset des filtres
-

MyRequestsComponent

Route : `/user/my-requests`

- `GET /api/pam/requests/mine` : liste de toutes les demandes de l'utilisateur
 - Formulaire de nouvelle demande :
 - `GET /api/pam/resources` → liste déroulante des ressources
 - Saisie justification et durée souhaitée
 - `POST /api/pam/requests`
 - Badges statut, lien vers la session si APPROVED
-

ActiveSessionsComponent

Route : `/user/sessions`

- `GET /api/pam/requests/active` : sessions APPROVED non expirées
 - Bouton de lancement par type :
 - SSH → `/user/terminal/{requestId}`
 - RDP → `/user/rdp/{requestId}`
 - WEB → `/user/web/{requestId}`
 - DATABASE → `/user/db/{requestId}`
 - Bouton "Terminer" → `PUT /api/pam/requests/{id}/terminate`
-

TerminalComponent

Route : `/user/terminal/:requestId`

- **xterm.js** : émulateur terminal dans le navigateur
- Addons : `FitAddon` (adaptation de la taille du terminal)

- Thème sombre GitHub-like (fond #0d1117 , curseur #58a6ff)

`ngAfterViewInit()` :

1. `initTerminal()` : instancie `Terminal` , ouvre dans `<div #terminalContainer>`
 2. `connectWebSocket()` :
 - `auth.getToken()` (Keycloak ou Face JWT)
 - Construit l'URL : `ws://localhost:8081/ws/session/{requestId}?token={jwt}`
 - `new WebSocket(url)`
 - `ws.onopen` → `connected = true` , message de bienvenue
 - `ws.onmessage` → `term.write(event.data)`
 - `ws.onerror` → affiche erreur
 - `ws.onclose` → affiche message de fin
 3. `term.onData(data)` → `ws.send(data)` : forward keystrokes
 4. `ResizeObserver` → `fitAddon.fit()` : ajustement dynamique
-

`RdpViewerComponent`

Route : `/user/rdp/:requestId`

- **guacamole-common-js** (chargé via CDN dans `index.html`)
- `declare const Guacamole: any` (API globale `Guacamole.js`)

`connectRdp()` :

1. `auth.getToken()`
 2. `new Guacamole.WebSocketTunnel(wsBase + '/ws/rdp/' + requestId)`
 3. `new Guacamole.Client(tunnel)`
 4. `client.getDisplay().getElement()` → ajouté au DOM
 5. `client.onstatechange` → gestion état (`CONNECTED=3` , `DISCONNECTED=5`)
 6. `new Guacamole.Mouse(containerRef)` → `sendMouseState()` sur `move/down/up/out`
 7. `new Guacamole.Keyboard(document)` → `sendKeyEvent()` sur `keydown/keyup`
 8. `ResizeObserver` → `client.sendSize(w, h)`
 9. `client.connect('token=' + jwt)` → le tunnel génère l'URL WebSocket finale
-

`WebViewerComponent`

Route : `/user/web/:requestId`

- Charge la ressource web dans une `<iframe>` via le proxy
 - URL : `/proxy/web/{requestId}/`
 - Note : `X-Frame-Options` désactivé côté backend pour permettre l'iframe
-

DbViewerComponent

Route : `/user/db/:requestId`

- Affiche les informations de connexion (host, port, database name, username)
 - Permet de copier l'URL de connexion JDBC/psql
 - Le tunnel TCP est géré côté backend
-

ProfileComponent

Route : `/profile`

- Informations du profil (username, e-mail, rôles, tenant)
 - Liens vers `/profile/mfa` et `/profile/face-verify`
-

MfaComponent

Route : `/profile/mfa`

Statut : GET `/api/user/mfa/status`

Enrollment :

1. Choix méthode (TOTP / EMAIL / SMS / WhatsApp)
2. Saisie e-mail / téléphone si EMAIL/SMS/WhatsApp
3. POST `/api/user/mfa/enroll/init` → affiche QR code (TOTP) ou message d'envoi
4. Saisie du code de confirmation
5. POST `/api/user/mfa/enroll/confirm { code }`

Suppression : DELETE `/api/user/mfa/enroll`

FaceVerifyComponent

Route : `/profile/face-verify`

- Accès caméra → flux vidéo en direct
 - Chargement des modèles face-api.js (SSD MobileNet + Face Landmark + Face Recognition)
 - Capture du descripteur → POST `/api/user/face/verify { descriptor[] }`
 - Retourne `{ match: boolean, distance: float }`
 - Affiché pour vérifier l'enrollment facial existant
-

7. Flux de données par fonctionnalité

7.1 Connexion Keycloak (Login standard)

Utilisateur → [Bouton "Se connecter"]

→ Angular KeycloakService.login()

→ Navigateur redirigé vers `http://localhost:8080/realms/iam-pam-realm/protocol/openid-connect?client_id=iam-pam-frontend&redirect_uri=http://localhost:3000/&response_type=code&scope=openid profile email`

Keycloak → page de login HTML

← Utilisateur saisit credentials (ou fédération LDAP)

Keycloak → redirige vers `http://localhost:3000/?code=XXXXX`

→ KeycloakService intercepte le code

→ POST `http://localhost:8080/realms/iam-pam-realm/protocol/openid-connect/token`
`{ code, client_id, redirect_uri, grant_type=authorization_code }`

← `{ access_token (JWT), refresh_token, id_token }`

Angular:

→ decode access_token → roles, groups, preferred_username

→ AuthGuard.isAccessAllowed() → rôle → redirect page d'accueil

7.2 Connexion Face ID

Utilisateur → [Saisit username + active caméra]

- `FaceService.loadModels()` → charge TensorFlow.js models depuis CDN
- `navigator.mediaDevices.getUserMedia({ video: true })`
- `FaceService.detectDescriptor(videoElement)`
 - `face-api.detectSingleFace(video).withFaceLandmarks().withFaceDescriptor()`
 - `Float32Array[128]`
- `POST /api/public/face/login`
`{ username: "alice", descriptor: [0.1, -0.2, ...128 valeurs...] }`

Backend (public, pas de JWT requis):

- `FaceDescriptorService.verifyFace(username, descriptor)`
 - `findAllByUsernameAndIsActiveTrue(username)` ← cross-tenant lookup
 - distance euclidienne = $\sqrt{\sum (d1[i] - d2[i])^2}$
 - match si distance < 0.5
- Si match:
 - récupérer tenant + rôles depuis profil stocké
 - `FaceTokenService.generateToken(username, roles, tenantId)`
 - JWT HS256 { sub, iss:"face-auth", roles, tenantId, groups:[tenantId] }
- ← { token, username, roles, tenantId, distance }

Angular:

- `faceAuth.saveSession(resp)` → `localStorage`
- redirect selon rôle

7.3 Vérification MFA post-login

Après login Keycloak:

- `MfaService.getTenantConfig()` → `mfaRequired: true`
- `MfaService.getStatus()` → `enrolled: true, method: "TOTP"`
- `redirect /mfa-verify`

Page `MfaVerify`:

Si `method == EMAIL` ou `SMS`:

- `POST /api/user/mfa/send-otp`
- Backend: `generateNumericOtp()` → `storeOtp("tenantId:username", code)`
 - `notificationService.sendEmailOtp(email, code)`
- ← `{ message: "Code envoyé à alice@bank-a.com" }`

Utilisateur saisit code → `POST /api/user/mfa/verify { code: "123456" }`

Backend:

- `enrollment.method == EMAIL`: `verifyStoredOtp("tenantId:username", code)`
 - `otpStore.get(key) = [hash, expire]`
 - `expire > now` ET `hash == code.hashCode()`
- Si valid: `enrollment.lastVerifiedAt = now`, `otpStore.remove(key)`
- ← `{ success: true, message: "Vérification réussie." }`

Angular → `redirect` vers page d'accueil

7.4 Enrollment TOTP

Utilisateur → [Choisit TOTP] → `POST /api/user/mfa/enroll/init { method: "TOTP" }`

Backend:

- `generateBase32Secret()` → 10 octets aléatoires → Base32 → ex: "JBSWY3DPEHPK3PXP"
- `pendingTotpSecrets["tenant:alice"] = secret`
- `buildTotpUri("alice", secret)`:
 - "otpauth://totp/IAM-PAM:alice?secret=JBSWY3DP...&issuer=IAM-PAM&algorithm=SHA1&digits=6&p"
- ← `{ method: TOTP, totpSecret: "JBSWY3DP...", totpQrUri: "otpauth://..." }`

Angular:

- Génère QR code depuis `totpQrUri` (bibliothèque `qrcode`)
- Utilisateur scanne avec Google Authenticator
- Saisit le code affiché → `POST /api/user/mfa/enroll/confirm { code: "482719" }`

Backend:

- `verifyTotp(secret, "482719")`
 - `T = floor(now/30) = 59423000`
 - Pour `T-1, T, T+1`: `HMAC-SHA1(base32decode(secret), T)` → troncature → mod 10^6
 - Si un code correspond → valid
- `enrollment.isActive = true`, `enrolledAt = now`
- ← `{ message: "Enrollment confirmé" }`

7.5 Créer et approuver une demande d'accès

```
Utilisateur → [Sélectionne ressource "Serveur SSH Prod", justification, durée 4h]
→ POST /api/pam/requests { resourceId: 42, justification: "Maintenance", durationHours: 4 }
Authorization: Bearer <jwt_user>

Backend TenantInterceptor:
→ JWT.groups[0] = "tenant-bank-a" → TenantContext.set("tenant-bank-a")

AccessRequestService.createRequest():
→ Charge Resource(42) → vérifie resource.tenantId == "tenant-bank-a"
→ AccessRequest { status: PENDING, requesterUsername: "alice", ... }
→ auditService.log(ACCESS_REQUESTED, "alice", "tenant-bank-a", "Serveur SSH Prod", 56, ...)
← { id: 56, status: "PENDING", ... }

---

Tenant-admin → [Voit la demande en attente, clique Approuver]
→ PUT /api/pam/requests/56/review { status: "APPROVED", comment: "OK pour maintenance" }
Authorization: Bearer <jwt_tenant_admin>

AccessRequestService.reviewRequest():
→ Charge AccessRequest(56) → status == PENDING
→ status = APPROVED
→ expiresAt = now + 4 heures
→ auditService.log(ACCESS_APPROVED, "tenant-admin", "tenant-bank-a", "Serveur SSH Prod", 56, ...)
← { id: 56, status: "APPROVED", expiresAt: "2026-05-08T18:30:00" }
```


7.6 Session SSH

Utilisateur → [Clique "Lancer Terminal" sur la session 56]

- Angular route: /user/terminal/56
- TerminalComponent.ngAfterViewInit()
- initTerminal() : Terminal xterm.js ouverte dans le DOM
- connectWebSocket():
 - token = await auth.getToken() // Keycloak JWT ou Face JWT
 - new WebSocket("ws://localhost:8081/ws/session/56?token=eyJhbGci...")

Upgrade WebSocket (HTTP → WS):

- Spring BastionTerminalHandler.afterConnectionEstablished()
 - Extraire ?token= → jwtDecoder.decode(token) → valide
 - extractRequestId("/ws/session/56") → 56L
 - sessionRequestMap.put(wsSession.getId(), 56L)
 - bastionService.openSession(56L, wsSession)

BastionService.openSession(56, wsSession):

- findByIdWithResource(56) → AccessRequest + Resource (JOIN FETCH)
- request.isActive() → true (status=APPROVED, pas expiré)
- resource: { host: "10.0.0.100", port: 22, credentialUsername: "ubuntu", credentialPassword: "s3cret" }
- sshClient.connect("bastion-agent", "192.168.112.138", 22)
- bastionSession.addPublicKeyIdentity(loadKeyPair("bastion_key"))
- bastionSession.auth().verify(15s)
- shell = bastionSession.createShellChannel()
- shell.setUsePty(true), shell.setPtyType("xterm-256color")
- Pipes: toShell (in), fromShellIn (out)
- shell.open()
- toShell.write("sshpass -p 's3cret' ssh -o StrictHostKeyChecking=no -p 22 ubuntu@10.0.0.100")
- auditService.log(SESSION_STARTED, ...)
- Thread démon: relayOutputToWs(shellOutput → wsSession)

Browser ← ws ← "ubuntu@server:~\$ " (prompt SSH)

Frappe utilisateur "ls -la\n":

- ws.send("ls -la\n")
- BastionTerminalHandler.handleTextMessage() → bastionService.sendInput(56, "ls -la\n")
- toShell.write("ls -la\n") → audit COMMAND_EXECUTED
- sshd exécute → sortie arrive dans shellOutput → relayOutputToWs → ws → xterm.js
- ← affichage de la liste de fichiers dans le terminal

Fermeture (navigateur ferme l'onglet):

- ws.onclose
- BastionTerminalHandler.afterConnectionClosed() → bastionService.closeSession(56)
- shell.close(), bastionSession.close()
- audit SESSION_ENDED

7.7 Session RDP

Utilisateur → [Clique "Ouvrir Bureau RDP" sur la session 57]

- Angular route: /user/rdp/57
- RdpViewerComponent.ngAfterViewInit()
- connectRdp():
 - token = await auth.getToken()
 - new Guacamole.WebSocketTunnel("ws://localhost:8081/ws/rdp/57")
 - new Guacamole.Client(tunnel)
 - display = client.getDisplay().getElement()
 - containerRef.nativeElement.appendChild(display)
 - Keyboard, Mouse handlers
 - client.connect("token=" + token)
 - WebSocket URL finale: ws://localhost:8081/ws/rdp/57?token=eyJhbGci...

Spring GuacamoleWebSocketHandler.afterConnectionEstablished():

- Valider JWT
- Thread: guacamoleService.openSession(57, wsSession)

GuacamoleService.openSession(57, wsSession):

- Charge AccessRequest(57) → Resource: { host: "10.0.0.200", port: 3389, ... }
- GuacamoleConfiguration:
 - protocol=rdp, hostname=10.0.0.200, port=3389,
 - username=Administrator, password=WinPass!, width=1280, height=720
- InetGuacamoleSocket("192.168.112.138", 4822) → connexion TCP à guacd
- ConfiguredGuacamoleSocket(socket, config)
 - handshake guacd: guacd reçoit la config, ouvre une connexion RDP vers 10.0.0.200
- SimpleGuacamoleTunnel(configuredSocket)
- audit SESSION_STARTED
- Thread: relayToWs(tunnel → wsSession)

guacd ← RDP → Windows Server 10.0.0.200

guacd → instructions Guacamole (rendu graphique)

relayToWs → wsSession.sendMessage(instruction)

Guacamole.js → rendu canvas HTML5 dans le navigateur

Clavier/souris utilisateur:

- Guacamole.js intercepte → instructions Guacamole (clé/souris)
- ws.send(instruction)
- handleTextMessage() → guacamoleService.sendInstruction(57, instruction)
- tunnel.acquireWriter().write(chars)
- guacd → RDP → Windows

Fermeture:

- tunnel.close()
- audit SESSION_ENDED

7.8 Session Web (Proxy HTTP)

Utilisateur → route /user/web/58

- WebViewComponent: <iframe src="/proxy/web/58/">
- GET /proxy/web/58/ (sans JWT – l'iframe ne peut pas envoyer de header)

Backend WebProxyService:

- Extrait requestId=58 du path
- Charge AccessRequest(58) → isActive() → true
- Resource: { host: "192.168.1.50", port: 8080 }
- HttpClient forward GET http://192.168.1.50:8080/
- Retourne le HTML au navigateur (réécriture des liens relatifs)

Navigations suivantes dans l'iframe: idem, toutes routées via /proxy/web/58/{subpath}

7.9 Expiration automatique des sessions

```
@Scheduled(fixedDelay = 60000) // Toutes les 60 secondes
```

```
AccessRequestService.expireRequests():
```

- findExpiredRequests(now, APPROVED)
SQL: SELECT * FROM access_requests WHERE status='APPROVED' AND expires_at < now()
- Pour chaque demande: status = EXPIRED
- saveAll(expired)
- Log INFO

Résultat: isActive() renvoie false → la prochaine tentative d'ouverture de session est refusée

8. Stockage des données

8.1 PostgreSQL — Base iam_pam_db

Schéma shared (tables partagées, filtrées par tenantId)

Table	Contenu	Colonnes clés
tenants	Registre des organisations	tenant_id, schema_name, domain, max_users, is_active
tenant_services	Abonnements services	tenant_id, service_type, is_active, expires_at
resources	Ressources PAM	name, type, host, port, credential_password (AES-256-GCM)

Table	Contenu	Colonnes clés
access_requests	Cycle de vie des demandes	requester_username , status , expires_at
audit_logs	Journal d'audit	action , username , resource_name , timestamp
tenant_mfa_config	Config MFA par tenant	Flags booléens, mfa_required
user_mfa_enrollment	Enrollments MFA	method , secret , is_active
user_face_descriptors	Descripteurs faciaux	descriptor_json (Float32[128])
tenant_ad_configs	Config LDAP/AD	server_url , bind_dn , bind_password

Schémas tenants (tenant_bank_a , tenant_bank_b , tenant_fintech_c)

Actuellement utilisés pour l'isolation future. Les tables applicatives sont toutes dans `shared` avec filtrage par `tenantId`.

8.2 PostgreSQL — Base `keycloak`

Gérée entièrement par Keycloak. Contient :

- Utilisateurs, credentials (hash bcrypt)
- Realm, clients, rôles, groupes
- Sessions OAuth2, tokens de refresh
- Configuration User Federation (LDAP)
- Mappers de claims JWT

Ne jamais accéder directement — utiliser l'API Admin Keycloak.

8.3 Mémoire vive (Spring Boot — non persistant)

Stockage	Type	Contenu	Durée
MfaService.otpStore	ConcurrentHashMap	OTP en cours (hash + expire)	5 minutes
MfaService.pendingTotpSecrets	ConcurrentHashMap	Secret TOTP en attente	Jusqu'à confirmation

Stockage	Type	Contenu	Durée
		de confirmation	
<code>BastionService.activeSessions</code>	<code>ConcurrentHashMap</code>	Sessions SSH actives (SSH session + pipes)	Durée de la connexion WebSocket
<code>GuacamoleService.activeSessions</code>	<code>ConcurrentHashMap</code>	Sessions RDP actives (tunnel Guacamole)	Durée de la connexion WebSocket
<code>BastionTerminalHandler.sessionRequestMap</code>	<code>ConcurrentHashMap</code>	Mapping wsSession.id → requestId	Durée de la connexion WebSocket
<code>GuacamoleWebSocketHandler.sessionRequestMap</code>	<code>ConcurrentHashMap</code>	Idem pour RDP	Durée de la connexion WebSocket

8.4 LocalStorage du navigateur

Utilisé uniquement pour l'authentification Face ID :

Clé	Contenu	Durée
<code>iam_face_token</code>	JWT HS256 signé	24 heures (configurable)
<code>iam_face_profile</code>	JSON { username, roles, tenantId, loginTime }	Jusqu'à <code>clearToken()</code>

8.5 Système de fichiers (Backend)

Fichier	Emplacement	Contenu
<code>bastion_key</code>	<code>src/main/resources/</code>	Clé privée RSA pour connexion SSH au bastion

Ce fichier est **gitignored** en production. Il doit être fourni via variable d'environnement `BASTION_KEY_PATH` ou monté comme secret Docker.

8.6 OpenLDAP

Persisté dans le volume Docker `openldap_data`.

Structure de l'annuaire :

```
dc=bank-a,dc=local
├── ou=users
│   ├── uid=alice (inetOrgPerson)
│   ├── uid=bob
│   └── ...
└── ou=groups
    └── cn=staff
```

Attributs synchronisés vers Keycloak : `uid` → `username`, `mail` → `email`, `givenName` → `firstName`, `sn` → `lastName`.

8.7 Keycloak (stocké dans PostgreSQL `keycloak`)

Concept	Stockage
Utilisateurs importés LDAP	Réplica dans <code>user_entity</code> + fédération live
Groupe tenants	<code>keycloak_group</code> (ex: <code>tenant-bank-a</code>)
Membres des groupes	<code>user_group_membership</code>
Rôles de realm	<code>keycloak_role</code>
Assignations rôles	<code>user_role_mapping</code>
Secrets clients	<code>client_secret</code> (chiffrés)

8.8 Flux de chiffrement des credentials

Création ressource (API):

```
request.credentialPassword = "s3cret"
```

```
    ↓ AesEncryptionConverter.convertToDatabaseColumn()
```

12 bytes IV (random) → AES-256-GCM encrypt → 7 bytes ciphertext + 16 bytes tag

→ Base64 → "AAAA...BBB...CCC=" stocké dans credential_password

Lecture ressource (API):

```
"AAAA...BBB...CCC=" depuis DB
```

```
    ↓ AesEncryptionConverter.convertToEntityAttribute()
```

Base64 decode → extraire IV (bytes 0-11) → AES-256-GCM decrypt

→ "s3cret" en mémoire uniquement (JAMAIS renvoyé dans la réponse API)

La réponse ResourceResponse contient seulement:

```
hasPassword: true, hasPrivateKey: false
```

Fin de la documentation technique — IAM/PAM System

Version 1.0 — Mai 2026